The ID3 Algorithm

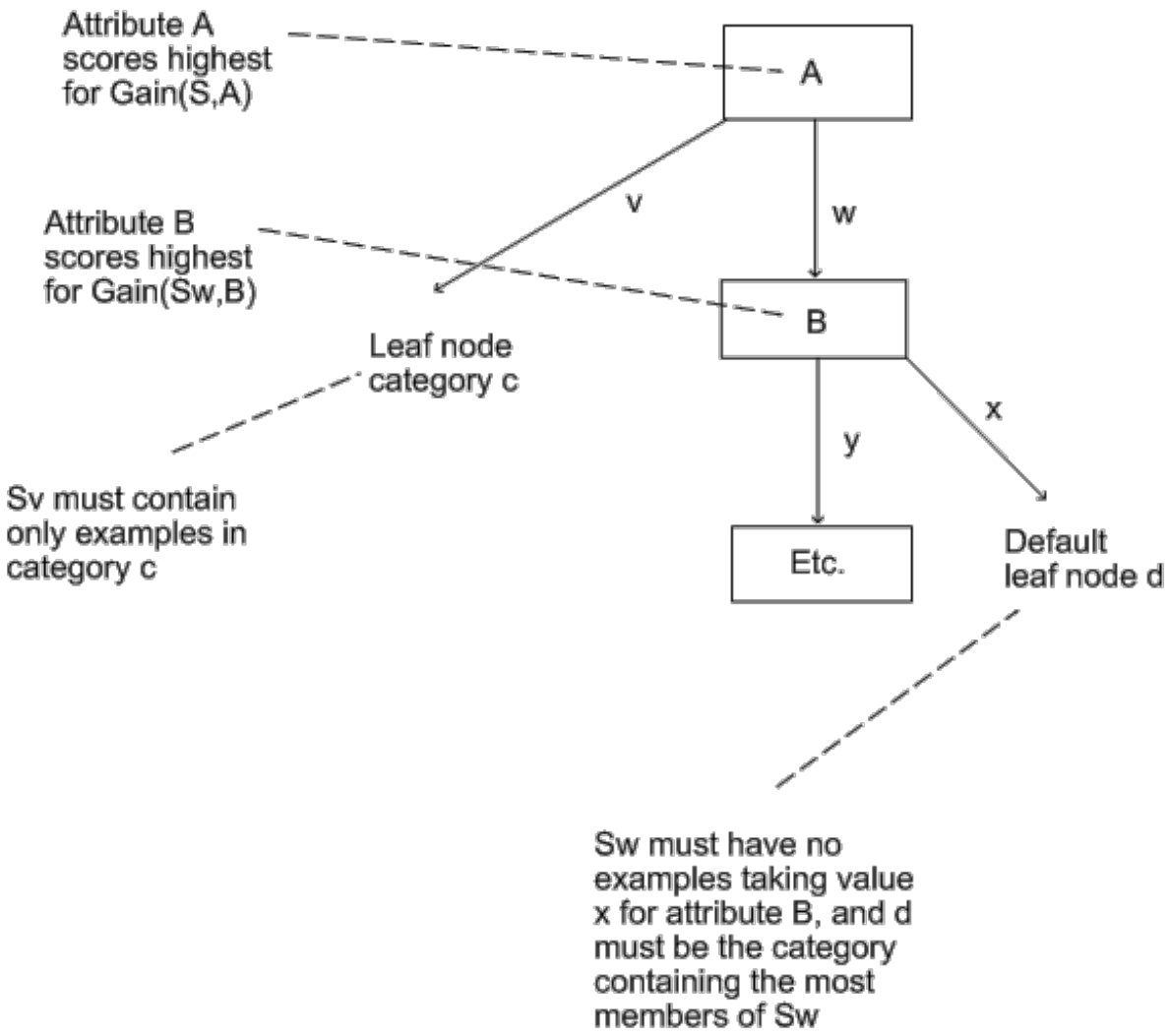
- Given a set of examples, S
  - Described by a set of attributes A_i
  - Categorised into categories c_j

1. Choose the root node to be attribute A
   - Such that A scores highest for information gain
     - Relative to S, i.e., gain(S,A) is the highest over all attributes
2. For each value v that A can take
   - Draw a branch and label each with corresponding v

The ID3 Algorithm

- For each branch you've just drawn (for value v)
  - If S_v only contains examples in category c
    - Then put that category as a leaf node in the tree
  - If S_v is empty
    - Then find the default category (which contains the most examples from S)
      - Put this default category as a leaf node in the tree
  - Otherwise
    - Remove A from attributes which can be put into nodes
    - Replace S with S_v
    - Find new attribute A scoring best for Gain(S, A)
    - Start again at part 2
- Make sure you replace S with S_v

Explanatory Diagram



A Worked Example

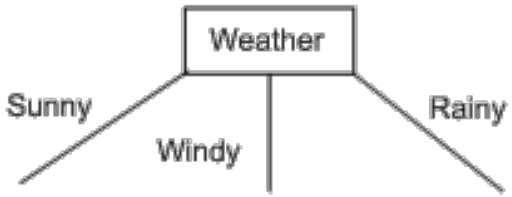
Weekend	Weather	Parents	Money	Decision (Category)
W1	Sunny	Yes	Rich	Cinema
W2	Sunny	No	Rich	Tennis
W3	Windy	Yes	Rich	Cinema
W4	Rainy	Yes	Poor	Cinema
W5	Rainy	No	Rich	Stay in
W6	Rainy	Yes	Poor	Cinema
W7	Windy	No	Poor	Cinema
W8	Windy	No	Rich	Shopping
W9	Windy	Yes	Rich	Cinema
W10	Sunny	No	Rich	Tennis

Information Gain for All of S

- S = {W1,W2,…,W10}
- Firstly, we need to calculate:
  - Entropy(S) = … = 1.571
- Next, we need to calculate information gain
  - For all the attributes we currently have available
    - (which is all of them at the moment)
  - Gain(S, weather) = … = 0.7
  - Gain(S, parents) = … = 0.61
  - Gain(S, money) = … = 0.2816
- Hence, the weather is the first attribute to split on
  - Because this gives us the biggest information gain

Top of the Tree

- So, this is the top of our tree:
- Now, we look at each branch in turn
  - In particular, we look at the examples with the attribute prescribed by the branch
- S_{sunny} = {W1,W2,W10}
  - Categorisations are cinema, tennis and tennis for W1,W2 and W10
  - What does the algorithm say?
    - Set is neither empty, nor a single category
    - So we have to replace S by S_{sunny} and start again



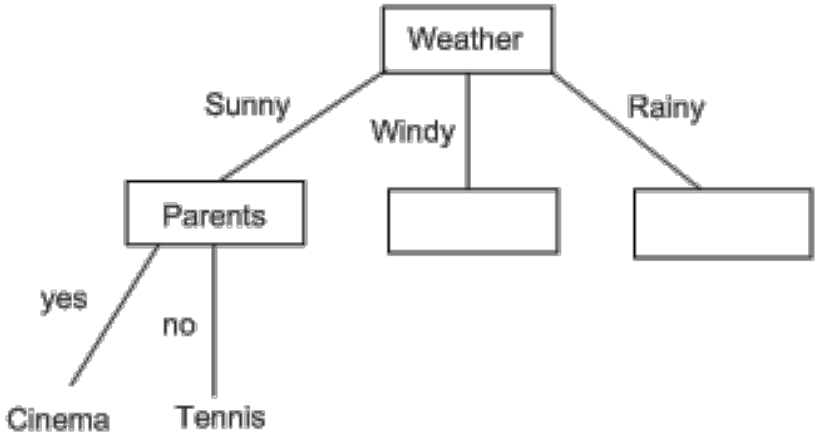
Working with S_{sunny}

Weekend	Weather	Parents	Money	Decision
W1	Sunny	Yes	Rich	Cinema
W2	Sunny	No	Rich	Tennis
W10	Sunny	No	Rich	Tennis

- Need to choose a new attribute to split on
  - Cannot be weather, of course – we've already had that
- So, calculate information gain again:
  - Gain(S_{sunny}, parents) = … = 0.918
  - Gain(S_{sunny}, money) = … = 0
- Hence we choose to split on parents

Getting to the leaf nodes

- If it's sunny and the parents have turned up
  - Then, looking at the table in previous slide
    - There's only one answer: go to cinema
- If it's sunny and the parents haven't turned up
  - Then, again, there's only one answer: play tennis
- Hence our decision tree looks like this:

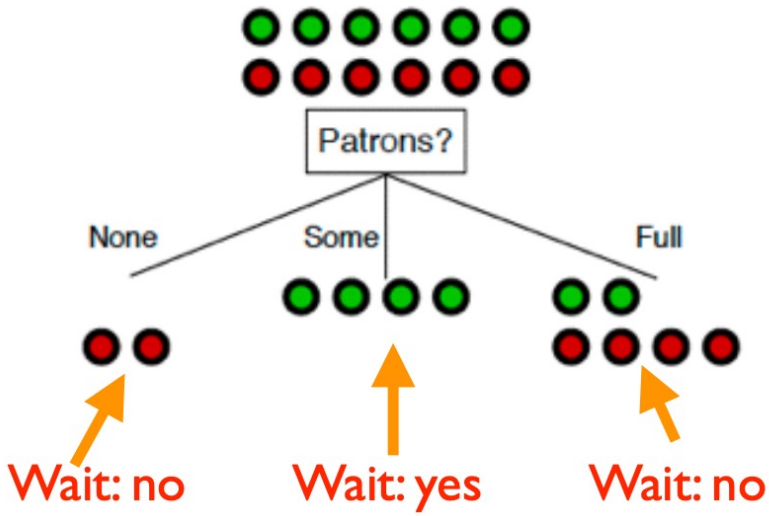


What is the optimal Tree Depth?

- We need to be careful to pick an appropriate tree depth.
- If the tree is too deep, we can overfit.
- If the tree is too shallow, we underfit
- Max depth is a hyper-parameter that should be tuned by the data. Alternative strategy is to create a very deep tree, and then to prune it.

Control the size of the tree

- If we stop early, not all training samples would be classified correctly.
- How do we classify a new instance:
  - We label the leaves of this smaller tree with the majority of training samples' labels



Summary of learning classification trees

- Advantages:
  - Easily interpretable by human (as long as the tree is not too big)
  - Computationally efficient
  - Handles both numerical and categorical data
  - It is parametric thus compact: unlike Nearest Neighborhood Classification, we do not have to carry our training instances around Building block for various ensemble methods (more on this later)
- Disadvantages
  - Heuristic training techniques
  - Finding partition of space that minimizes empirical error is NP-hard.
  - We resort to greedy approaches with limited theoretical underpinning.

Feature Space

- Suppose that we have p explanatory variables X1, . . . , Xp and n observations.

  – a numeric variable: n − 1 possible splits
  – an ordered factor: k − 1 possible splits
  – an unordered factor: −→ 2(k−1) − 1 possible splits.

Measures of Impurity

- At each node *i* of a classification tree, we have a probability distribution p_{ik} over *k* classes.

$$\hat{p}_{ik} = \frac{n_{ik}}{n_{i.}}$$

- Deviance: $D = \sum D_i$, where $D_i = -2 \sum_k n_{ik} \log(p_{ik})$.
- Entropy: $\sum p_{ik} \log(p_{ik})$.
- Gini index: $\sum_{j \neq k} p_{ij} p_{ik} = 1 - \sum_k p_{ik}^2$.
- Residual sum of squares

$$D = \sum_{\text{cases } j} \left(y_j - \mu_{[j]} \right)^2$$

where $\mu_{[j]}$ is the mean of the values in the node that case j belongs to.

Pruning Rules

- Stop when one instance in each leaf (regression problem)
- Stop when all the instance in each leaf have the same label (classification problem)
- Stop when the number of samples in each leaf is less than the threshold
- Stop when the leaf's error is less than the threshold
- Stop when the number of instances in each leaf is less than the threshold
- Stop when the p-value between two divided leaves is larger than the certain threshold (e.g. 0.05 or 0.01) based on chosen statistical tests.

Thanks.

Machine Learning by Tom Mitchell